

Lab 3 – Architectural Justification

Problem Statement #41 — OpenAPI Mock Server Generator

Architecture Selection: We chose a Layered Architecture (Presentation, Business, and Data layers) for the OpenAPI Mock Server Generator system.

Reason 1: Matches the natural request pipeline

The system's core requirements (FR-001–FR-005) describe a strictly sequential pipeline: a spec is uploaded and parsed, mock endpoints are generated, incoming requests are validated against the schema, and results are logged. A layered style maps each stage onto a distinct layer, the Presentation layer handles API Developer, QA Engineer, and CI/CD Pipeline traffic; the Business layer owns spec parsing, validation, and mock-response generation; the Data layer owns persistence. This keeps each Epic (Spec Ingestion, Request Validation, Testing & Observability) isolated to one or two layers, so teams can build and test them independently without touching unrelated code.

Reason 2: Simplifies support for multiple client types

API Developers (via UI/CLI), QA Engineers, and an automated CI/CD Pipeline all consume the same mock server through the Presentation layer's API Gateway. Because the layered style forces all external interaction through one entry point, new client types or protocol changes only affect that top layer, the mock-generation and validation logic underneath remains untouched, unlike a microservices split where each client integration could ripple across independently deployed services.

Security Advantage

NFR-002 requires that all stored OpenAPI specs and request/response logs be encrypted at rest (AES-256). In this layered design, only the Data layer's Secure Data Store component ever touches raw storage, the Spec Parser & Validator and Request Logger components access it exclusively through the ISpecStorage and ILogStorage interfaces. This means encryption is enforced in a single, auditable place, and no upper-layer component (including the API Gateway that is directly exposed to external actors) can accidentally read or write unencrypted data.

Performance Benefit

NFR-001 requires the mock engine to sustain 1,000 requests/second with latency accurate to ± 10 ms. Because schema validation and mock-response generation are confined to in-memory Business-layer components (Spec Parser & Validator and Mock Response Generator), the hot request path never blocks on disk I/O, only the asynchronous Request Logger writes to the encrypted Data layer. This lets the Business layer be scaled or optimized (e.g., caching parsed schemas) independently of storage performance, directly supporting the throughput and latency targets.

OpenAPI Mock Server Generator — UML Component Diagram

Layered Architecture (Presentation / Business / Data)

